

PSE-TRAVERSE-SIGNATURE-01

Domänenagnostische Spezifikation für Operator-Signatur-Traversierung

Extraktion des neutralen Spektral-Mining-Kerns aus ZetaLab für `pse-traverse`

Status: Implementierungs-Spezifikation / Blueprint v0.1
Sprache: Deutsch, normative technische Fassung
Zielsystem: `pse-traverse` im PSE-Ökosystem
Nichtziel: Riemann-, Zeta-, Theta- oder RH-Forschung im Kern
Autor: Sebastian Klemm / technische Ausarbeitung durch GPT-5.5 Thinking
Datum: 7. Mai 2026

Dieses Dokument ist als direkte Implementierungsgrundlage formuliert. Es enthält ausschließlich Architektur-, Datenmodell-, Algorithmus-, Gate-, Replay-, Test- und Integrationsanforderungen für eine domänenagnostische Signatur-Traversierungsschicht. Alle historischen oder domänenspezifischen Begriffe aus der Zeta-/Riemann-Forschung sind aus dem Kern entfernt und erscheinen nur als ausdrücklich ausgeschlossene Nichtziele.

Inhaltsverzeichnis

1	Zweck und Geltungsbereich	4
1.1	Primäre Ziele	4
1.2	Nichtziele	4
2	Normative Sprachregeln	4
3	Architekturposition im PSE-System	5
3.1	Beziehung zu vorhandenen Komponenten	5
4	Begriffsmodell	5
4.1	Struktur	5
4.2	Operator	6
4.3	Signatur	6
4.4	Diagnose	6
4.5	Regime	6
5	Domänenneutralisierung	6
6	Modulzuschnitt	7
6.1	Exportanforderung	7
7	Datenmodell	7
7.1	CanonicalValue und SignatureValue	7
7.2	StructuralOperator	8
7.3	MatrixProfile	8
7.4	Signature	8
7.5	SignatureSummary	9
7.6	SignatureDiagnostics	9
8	Operatorbildung	10
8.1	Input	10
8.2	Laplacian-Profil	10
8.3	Directed Coupling Profile	10
8.4	Constraint-Potential	10
8.5	Composite Operator	11
9	Signaturerzeugung	11
9.1	Minimalalgorithmus v0.1	11
9.2	Exakte Kleinmatrix-Option	11
10	Signaturdiagnostik	11
10.1	Score-Definitionen	11
10.1.1	Fragmentation Score	12
10.1.2	Asymmetry Score	12
10.1.3	Degeneracy Score	12
10.1.4	Gap Score	12
10.1.5	Rigidity Score	12
10.2	Regime-Klassifikation	12

11 Signature Gate	13
11.1 Funktion	13
11.2 Typen	13
11.3 Fail-Closed-Regel	13
12 Blueprint Search	13
12.1 Zweck	13
12.2 TraversalBlueprint	13
12.3 Search Policies	14
12.4 Nichtdominierte Frontier	14
13 Search Ledger	14
13.1 Funktion	14
13.2 SearchLedgerEntry	14
13.3 Hash Chain	15
14 Search Autopilot	15
14.1 Phasenmodell	15
14.2 AutopilotConfig	15
14.3 Stop-Regeln	16
15 Integration in Reports und Zertifikate	16
15.1 TraversalRunReport-Erweiterung	16
15.2 CollapseCertificate-Erweiterung	16
15.3 GateReport-Erweiterung	16
16 CLI-Anforderungen	17
16.1 Subcommand: plan	17
16.2 Subcommand: run	17
16.3 Subcommand: search	17
17 Determinismus und Canonicalization	17
17.1 Kanonische Serialisierung	17
17.2 Sortierpflicht	17
17.3 Seeds	18
18 Fehlersemantik	18
18.1 Fehlertypen	18
18.2 Fehlerauswirkung	18
19 Testanforderungen	18
19.1 Unit Tests	18
19.2 Integration Tests	19
20 Definition of Done	19
21 Empfohlene Implementierungssequenz	19
21.1 Phase S1 - Typen und Canonicalization	19
21.2 Phase S2 - Operatorbildung aus DoFGraph	20
21.3 Phase S3 - Signaturdiagnostik	20
21.4 Phase S4 - Signature Gate	20
21.5 Phase S5 - Blueprint Search und Ledger	20
21.6 Phase S6 - Autopilot und Benchmark	20

22 Beispiel: neutrale Problemverarbeitung	20
22.1 Input	20
22.2 Operator	21
22.3 Diagnose	21
22.4 Gate-Wirkung	21
23 Sicherheits- und Missbrauchsgrenzen	21
24 Publikations- und Benennungsdisziplin	21
25 Zusammenfassung der Implementierungslogik	21
A Agenten-Patch-Briefing	22

1 Zweck und Geltungsbereich

Diese Spezifikation definiert eine optionale Erweiterung von **pse-traverse**, die strukturierte Problemräume über Operatorbildung in Signaturen überführt, diese Signaturen diagnostiziert und die resultierenden Messwerte für Gate-Entscheidungen, Plan-Ranking, Benchmarking, Suchsteuerung und Zertifizierung nutzt.

Die Erweiterung wird als **Operator-Signatur-Traversierung** bezeichnet. Sie ist kein mathematisches Spezialmodul, kein Riemann-Hypothese-Modul, kein Zeta-Nullstellen-Miner und kein Domänenadapter. Sie ist ein neutraler postsymbolischer Mechanismus:

Struktur → **Operator** → **Signatur** → **Diagnose** → **Gate** → **Suche** → **Evidenz**

1.1 Primäre Ziele

Die Implementierung **MUSS** folgende Ziele erfüllen:

1. Ein **DoFGraph** oder ein äquivalenter strukturierter Traversierungsraum **MUSS** in einen neutralen **StructuralOperator** übersetzbar sein.
2. Ein **StructuralOperator** **MUSS** eine deterministische **Signature** erzeugen.
3. Eine **Signature** **MUSS** durch **SignatureDiagnostics** bewertet werden.
4. Die Diagnosewerte **MUSS** in **GateReport**, **CollapseCertificate** und **TraversalRunReport** referenzierbar sein.
5. Alle neuen Artefakte **MUSS** kanonisch serialisiert und content-addressed sein.
6. Der Kern **MUSS** vollständig domänenagnostisch bleiben.
7. Der Kern **MUSS** ohne Netzwerkzugriff und ohne Python-/NumPy-/SciPy-Abhängigkeit kompilieren.
8. Echte **SemanticCrystals** **MUSS** weiterhin ausschließlich über die existierende PSE-Commit-Brücke entstehen.

1.2 Nichtziele

Die Implementierung **DARF NICHT** folgende Dinge tun:

1. Keine Begriffe wie **zeta**, **riemann**, **theta**, **RH**, **GUE**, **GOE**, **Poisson** als Kern-Domänenbegriffe einführen.
2. Keine Riemann-Hypothese- oder Zeta-Nullstellen-Metrik im Kern implementieren.
3. Keine numerische Behauptung als mathematischen Beweis ausgeben.
4. Keine Signaturdiagnose als Ersatz für PSE-Gate, Domain-Verifier oder Human-Review behandeln.
5. Keine **SemanticCrystals** synthetisch erzeugen.
6. Keine nichtdeterministische Zufallsquelle ohne expliziten Seed verwenden.
7. Keine Wall-Clock-Zeit in content-addressed Identitäten aufnehmen.

2 Normative Sprachregeln

Die Begriffe **MUSS**, **DARF NICHT**, **SOLLTE** und **DARF** sind normativ.

Begriff	Bedeutung
MUSS	Verbindliche Anforderung. Ein Verstoß ist ein Implementierungsfehler.
DARF NICHT	Verbindliches Verbot. Ein Verstoß ist ein Spezifikationsbruch.

SOLLTE	Starke Empfehlung. Abweichungen müssen begründet und getestet werden.
DARF	Zulässige Option ohne Verpflichtung.

3 Architekturposition im PSE-System

Die Signatur-Schicht liegt zwischen Traversierungsplanung und Gate-/Report-Schicht. Sie verändert die bestehende PSE-Architektur nicht, sondern liefert zusätzliche Mess- und Suchsignale.

Listing 1: Zielpipeline mit Signatur-Schicht

```

ProblemSpec
-> FieldCube
-> DoFGraph
-> CollapsePlan
-> StructuralOperator
-> Signature
-> SignatureDiagnostics
-> Candidate
-> GateReport
-> CommitOutcome
-> CollapseCertificate
-> TraversalRunReport

```

3.1 Beziehung zu vorhandenen Komponenten

Vorhandene Komponente	Beziehung zur Signatur-Schicht
ProblemSpec	Quelle des fachlichen Problemraums. Wird nicht durch Signatur-Schicht verändert.
FieldCube	Liefert Dimensionen, Constraints, Couplings, EvidenceRefs und Topologie-Informationen.
DoFGraph	Primärer Input für Operatorbildung.
CollapsePlan	Kann durch Signaturdiagnosen gerankt, validiert oder refinanziert werden.
GateReport	Nimmt optionale SignatureGateOutcomes auf.
CommitOutcome	Bleibt unverändert: Crystal, NoCrystal, EvidenceOnly, GateFailed.
CollapseCertificate	Referenziert Signatur- und Diagnoseadressen als Beweisteile.
PseMacroStepCommitter	Bleibt einzige legitime Brücke zu SemanticCrystal.

4 Begriffsmodell

4.1 Struktur

Eine Struktur ist ein gerichteter oder ungerichteter, gewichteter oder ungewichteter Zusammenhang aus Knoten, Kanten, Constraints, Evidenzreferenzen und optionalen Pfaden. Im bestehenden System ist der wichtigste Strukturträger der DoFGraph.

4.2 Operator

Ein Operator ist eine deterministisch erzeugte, kanonisch beschriebene Transformation einer Struktur in eine auswertbare mathematische Form. In v0.1 wird der Operator als Matrixprofil modelliert, nicht zwingend als materialisierte Matrix.

4.3 Signatur

Eine Signatur ist ein geordneter Vektor oder eine geordnete Zusammenfassung von Messwerten, die aus einem Operator abgeleitet wurden. Sie ist kein Symboltext, keine semantische Beschreibung und kein Domänenurteil. Sie ist ein messbarer Abdruck der Struktur.

4.4 Diagnose

Eine Diagnose ist eine deterministische Auswertung einer Signatur. Sie erzeugt Scores, Regime-Hinweise und Gate-relevante Befunde.

4.5 Regime

Ein Regime ist eine neutrale Strukturklasse. Es beschreibt nicht die Domäne, sondern den Kopplungscharakter einer Struktur.

Regime	Bedeutung
Uncoupled	Schwach gekoppelte oder fragmentierte Struktur.
SymmetricCoupled	Weitgehend symmetrisch gekoppelte Struktur ohne starke Richtungsvorzugsordnung.
DirectedCoupled	Struktur mit relevanter gerichteter oder kausaler Kopplung.
Hierarchical	Mehrschichtige oder modular verschachtelte Struktur.
Degenerate	Struktur mit hoher Mehrdeutigkeit, Symmetrie, Nullraumgröße oder nicht unterscheidbaren Freiheitsgraden.
Unknown	Diagnose reicht nicht aus oder ist numerisch instabil.

5 Domänenneutralisierung

Alle aus ZetaLab stammenden Muster werden neutralisiert. Die historische Herkunft darf in Implementierungskommentaren nicht als Kernsemantik erscheinen.

Historischer Begriff	Zulässiger neutraler Kernbegriff
Zeta zero	TargetSignature oder TargetEvent, nur in Adaptern
Zeta alignment	SignatureAlignment
Wasserstein distance to Zeta	TargetDistanceMetric
GUE/GOE/Poisson	DirectedCoupled/SymmetricCoupled/Uncoupled
Hilbert-Pólya operator	CandidateOperator oder StructuralOperator
Flux Hamiltonian	DirectedCouplingOperator
Gamma / magnetic flux	AsymmetryStrength
Research Autopilot	SearchAutopilot
Blueprint mining	BlueprintSearch
Ledger	SearchLedger

Pareto frontier	NonDominatedFrontier
Coagula gate	FailClosedGate

6 Modulzuschnitt

Die Implementierung **MUSS** in klar getrennten Modulen erfolgen. Die nachfolgenden Dateinamen sind Zielnamen. Wenn die Codebasis bestehende Konventionen vorgibt, **DARF** die Benennung angepasst werden, sofern die öffentliche API semantisch identisch bleibt.

Listing 2: Zielmodule

```
crates/pse-traverse/src/operator.rs
crates/pse-traverse/src/signature.rs
crates/pse-traverse/src/signature_diag.rs
crates/pse-traverse/src/blueprint_search.rs
crates/pse-traverse/src/search_ledger.rs
crates/pse-traverse/src/search_autopilot.rs
crates/pse-traverse/src/signature_gate.rs
```

6.1 Exportanforderung

lib.rs **MUSS** die folgenden Typen öffentlich exportieren:

```
pub mod operator;
pub mod signature;
pub mod signature_diag;
pub mod blueprint_search;
pub mod search_ledger;
pub mod search_autopilot;
pub mod signature_gate;

pub use operator::{StructuralOperator, OperatorKind, MatrixProfile, OperatorBuildConfig};
pub use signature::{Signature, SignatureSummary, SignatureValue};
pub use signature_diag::{SignatureDiagnostics, RegimeHint, DiagnosticConfig};
pub use blueprint_search::{TraversalBlueprint, BlueprintSearchPolicy};
pub use search_ledger::{SearchLedgerEntry, SearchLedgerReport};
pub use search_autopilot::{SearchAutopilot, AutopilotPhase, AutopilotConfig};
pub use signature_gate::{SignatureGateConfig, SignatureGateOutcome};
```

7 Datenmodell

7.1 CanonicalValue und SignatureValue

Floating-Point-Werte sind deterministisch problematisch. Der Kern **MUSS** für content-addressed Artefakte eine kanonische Zahlendarstellung verwenden.

Listing 3: Kanonische Zahlendarstellung

```
pub struct SignatureValue {
    pub mantissa: i64,
    pub scale: i32,
}
```

Semantik:

$$\text{value} = \text{mantissa} \cdot 10^{-\text{scale}} \quad (1)$$

Implementierungsvorgabe:

1. Eingehende f64-Werte **MUSS** vor Speicherung quantisiert werden.
2. Die Quantisierung **MUSS** in der Konfiguration stehen.
3. NaN, +Inf und -Inf **MUSS** Gate-Failure oder Diagnose-Failure erzeugen und **DARF NICHT** in content-addressed Signaturen erscheinen.

7.2 StructuralOperator

Listing 4: StructuralOperator

```
pub struct StructuralOperator {
    pub operator_id: Hash256,
    pub substrate_id: Hash256,
    pub kind: OperatorKind,
    pub matrix_profile: MatrixProfile,
    pub parameters: BTreeMap<String, SignatureValue>,
    pub evidence_refs: Vec<EvidenceRef>,
    pub operator_version: String,
}

pub enum OperatorKind {
    Laplacian,
    DirectedCoupling,
    ConstraintWeighted,
    Composite,
}
```

`operator_id` **MUSS** als SHA-256 über die JCS-kanonische Darstellung aller reproduktionsrelevanten Felder berechnet werden. `operator_id` **DARF NICHT** aus Wall-Clock-Zeit, Pfaden, Hostnamen oder nicht-reproduzierbaren Metadaten abgeleitet werden.

7.3 MatrixProfile

Listing 5: MatrixProfile

```
pub struct MatrixProfile {
    pub n: usize,
    pub diagonal: Vec<SignatureValue>,
    pub symmetric_edges: Vec<WeightedEdge>,
    pub directed_edges: Vec<WeightedDirectedEdge>,
    pub constraint_weights: Vec<NodeWeight>,
}
```

Alle Vektoren **MUSS** stabil sortiert sein:

- `symmetric_edges`: nach `min(u,v)`, dann `max(u,v)`, dann Gewicht.
- `directed_edges`: nach `from`, dann `to`, dann Gewicht.
- `constraint_weights`: nach `node`, dann Gewicht.

7.4 Signature

Listing 6: Signature

```
pub struct Signature {
    pub signature_id: Hash256,
    pub operator_id: Hash256,
    pub values: Vec<SignatureValue>,
    pub summary: SignatureSummary,
    pub algorithm: SignatureAlgorithm,
    pub algorithm_version: String,
}

pub enum SignatureAlgorithm {
    MatrixProfileApprox,
    ExactSmallMatrix,
    PowerIterationProfile,
}
```

v0.1 **DARF** ohne vollständige Eigenwertzerlegung starten, sofern eine deterministische approximative Profilierung implementiert wird. Für kleine Matrizen **SOLLTE** eine exakte, deterministische Jacobi- oder QR-Variante implementiert werden, wenn dies ohne externe nichtdeterministische Bibliothek möglich ist.

7.5 SignatureSummary

Listing 7: SignatureSummary

```
pub struct SignatureSummary {
    pub dimension: usize,
    pub min_value: SignatureValue,
    pub max_value: SignatureValue,
    pub mean_value: SignatureValue,
    pub mean_spacing: SignatureValue,
    pub zero_multiplicity: usize,
    pub gap_profile: Vec<SignatureValue>,
}
```

7.6 SignatureDiagnostics

Listing 8: SignatureDiagnostics

```
pub struct SignatureDiagnostics {
    pub diagnostics_id: Hash256,
    pub signature_id: Hash256,
    pub gap_score: f64,
    pub degeneracy_score: f64,
    pub rigidity_score: f64,
    pub asymmetry_score: f64,
    pub fragmentation_score: f64,
    pub regime_hint: RegimeHint,
    pub passed_numeric_sanity: bool,
    pub messages: Vec<String>,
}

pub enum RegimeHint {
    Uncoupled,
    SymmetricCoupled,
    DirectedCoupled,
    Hierarchical,
}
```

```

    Degenerate,
    Unknown,
}

```

Die Scores **MUSS** im Intervall $[0, 1]$ liegen. Werte außerhalb des Intervalls sind Implementierungsfehler, außer sie werden vor Speicherung strikt geclamped und als Diagnosewarnung dokumentiert.

8 Operatorbildung

8.1 Input

Der Operator-Builder **MUSS** mindestens einen **DoFGraph** akzeptieren. Optional **DARF** ein **CollapsePlan** einbezogen werden, wenn Plan-Reihenfolge, Excision-Entscheidungen oder Gate-Gewichte die Operatorbildung beeinflussen sollen.

```

pub fn build_operator_from_dof_graph(
    graph: &DoFGraph,
    config: &OperatorBuildConfig,
) -> Result<StructuralOperator, OperatorBuildError>;

```

8.2 Laplacian-Profil

Für einen ungerichteten gewichteten Graphen $G = (V, E, w)$ gilt:

$$L = D - A \quad (2)$$

mit:

$$D_{ii} = \sum_j A_{ij}, \quad A_{ij} = w(i, j) \quad (3)$$

Die Implementierung **MUSS** den Laplacian nicht zwingend als dichte Matrix materialisieren. Ein **MatrixProfile** reicht aus, solange Signatur und Diagnostik deterministisch ableitbar sind.

8.3 Directed Coupling Profile

Gerichtete Kopplungen werden über ein antisymmetrisches Feld modelliert:

$$A_{ij}^{dir} = c(i \rightarrow j) - c(j \rightarrow i) \quad (4)$$

Dieses Feld ist neutral. Es beschreibt keine Physik, keinen Magnetismus und keine Spezialdomäne. Es beschreibt lediglich asymmetrische Abhängigkeit.

8.4 Constraint-Potential

Hard- und Soft-Constraints **DARF** als diagonales Gewichtungsprofil einfließen:

$$V_K(i, i) = \lambda_h h_i + \lambda_s s_i \quad (5)$$

wobei h_i die normalisierte Hard-Constraint-Belastung und s_i die normalisierte Soft-Constraint-Belastung eines Knotens bezeichnet.

8.5 Composite Operator

Ein zusammengesetzter Operator **DARF** als neutrales Profil definiert werden:

$$H = L + V_K + \gamma A^{dir} \quad (6)$$

In der Implementierung **MUSS** dies nicht als komplexe Matrix erscheinen. Entscheidend ist, dass `asymmetry_score` und gerichtete Kopplungsdiagnosen reproduzierbar aus demselben Profil entstehen.

9 Signaturerzeugung

9.1 Minimalalgorithmus v0.1

v0.1 **MUSS** einen Algorithmus bereitstellen, der ohne externe numerische Schwergewichtsabhängigkeiten reproduzierbare Signaturen erzeugt. Der Minimalalgorithmus berechnet:

1. Knotengrade und gewichtete Grade.
2. Sortierte Diagonalwerte des Laplacian-Profiles.
3. Approximate Gap Profile aus sortierten Diagonal- und Edge-Statistiken.
4. Nullmultiplikator-Schätzung über Komponentenanzahl.
5. Asymmetrieprofil aus gerichteten Kanten.

Listing 9: Minimaler Signatur-Algorithmus

```
pub fn derive_signature(
    operator: &StructuralOperator,
    config: &SignatureConfig,
) -> Result<Signature, SignatureError> {
    // 1. Validate finite canonical values
    // 2. Build deterministic profile vector
    // 3. Sort values canonically
    // 4. Quantize
    // 5. Compute summary
    // 6. Content-address signature
}
```

9.2 Exakte Kleinmatrix-Option

Für $n \leq N_{exact}$ **SOLLTE** eine exakte Spektralroutine implementiert werden. Diese Routine **MUSS** deterministisch sein und **MUSS** plattformbedingte Nichtdeterminismen minimieren.

Falls exakte Eigenwerte implementiert werden, gelten:

1. Ergebniswerte **MUSS** sortiert werden.
2. Vor Content Addressing **MUSS** quantisiert werden.
3. Iterationslimits **MUSS** fest konfiguriert sein.
4. Nicht-Konvergenz **MUSS** einen expliziten Diagnosefehler erzeugen.

10 Signaturdiagnostik

10.1 Score-Definitionen

Die folgenden Scores sind in v0.1 verbindlich.

10.1.1 Fragmentation Score

$$F = \min \left(1, \frac{c - 1}{\max(1, n - 1)} \right) \quad (7)$$

wobei c die Anzahl schwach zusammenhängender Komponenten und n die Knotenzahl ist. $F = 0$ bedeutet keine Fragmentierung, $F = 1$ maximale Fragmentierung.

10.1.2 Asymmetry Score

$$A_s = \frac{\sum_{i,j} |A_{ij}^{dir}|}{\sum_{i,j} |A_{ij}^{sym}| + \sum_{i,j} |A_{ij}^{dir}| + \epsilon} \quad (8)$$

$A_s = 0$ bedeutet keine gerichtete Kopplung; $A_s \rightarrow 1$ bedeutet dominante gerichtete Kopplung.

10.1.3 Degeneracy Score

$$D = \frac{m_{dup}}{\max(1, n)} \quad (9)$$

wobei m_{dup} die Anzahl nicht-eindeutiger Signaturwerte nach Quantisierung bezeichnet. Hohe Degeneration kann auf strukturelle Mehrdeutigkeit oder zu grobe Modellierung hinweisen.

10.1.4 Gap Score

$$G = \min \left(1, \frac{\Delta_2}{\Delta_{ref} + \epsilon} \right) \quad (10)$$

Δ_2 ist der erste relevante positive Abstand im Gap-Profil. Δ_{ref} ist eine konfigurierbare Referenzskala.

10.1.5 Rigidity Score

$$R = 1 - \min \left(1, \frac{\text{var}(gaps)}{\text{mean}(gaps)^2 + \epsilon} \right) \quad (11)$$

Hohe Starrheit bedeutet gleichmäßigere Abstände und global stabilere Signaturstruktur. Niedrige Starrheit bedeutet unruhige oder fragile Signaturstruktur.

10.2 Regime-Klassifikation

Die Regime-Klassifikation **MUSS** regelbasiert, deterministisch und transparent sein.

Listing 10: Regime-Klassifikation v0.1

```

if fragmentation_score >= cfg.fragmentation_high {
    RegimeHint::Uncoupled
} else if degeneracy_score >= cfg.degeneracy_high {
    RegimeHint::Degenerate
} else if asymmetry_score >= cfg.asymmetry_high {
    RegimeHint::DirectedCoupled
} else if rigidity_score >= cfg.rigidity_high && asymmetry_score < cfg.asymmetry_low {
    RegimeHint::SymmetricCoupled
} else if hierarchy_indicator >= cfg.hierarchy_high {
    RegimeHint::Hierarchical
} else {
    RegimeHint::Unknown
}

```

11 Signature Gate

11.1 Funktion

Das Signature Gate ist ein zusätzlicher Gate-Kanal. Es **DARF NICHT** das bestehende Fail-Closed-Gate ersetzen. Es **MUSS** entweder:

1. zusätzliche Fail-Gründe liefern,
2. Plan-Ranking beeinflussen,
3. Benchmark-Scores liefern,
4. oder als EvidenceRef in Zertifikaten erscheinen.

11.2 Typen

Listing 11: SignatureGate

```
pub struct SignatureGateConfig {
    pub enabled: bool,
    pub min_gap_score: f64,
    pub max_degeneracy_score: f64,
    pub max_fragmentation_score: f64,
    pub allowed_regimes: Vec<RegimeHint>,
    pub fail_closed: bool,
}

pub struct SignatureGateOutcome {
    pub outcome_id: Hash256,
    pub diagnostics_id: Hash256,
    pub passed: bool,
    pub reasons: Vec<String>,
    pub measured: BTreeMap<String, f64>,
    pub thresholds: BTreeMap<String, f64>,
}
```

11.3 Fail-Closed-Regel

Wenn `SignatureGateConfig.enabled=true` und `fail_closed=true`, dann **MUSS** ein nicht bestandenenes Signature Gate den `GateReport.passed=false` setzen oder einen eigenen Fail-Channel im `GateReport` aktivieren, der den Commit verhindert.

Wenn `fail_closed=false`, **DARF** das Ergebnis nur als Diagnose oder Ranking-Signal einfließen. Diese Einstellung **MUSS** im `TraversalRunDescriptor` und im `TraversalRunReport` sichtbar sein.

12 Blueprint Search

12.1 Zweck

Die Blueprint-Suche erzeugt mehrere operative Traversierungsvarianten für denselben `ProblemSpec`. Damit wird das System von einer Einzelpipeline zu einer kontrollierten Suchmaschine über Plan- und Operatorvarianten.

12.2 TraversalBlueprint

Listing 12: TraversalBlueprint

```
pub struct TraversalBlueprint {
    pub blueprint_id: Hash256,
    pub problem_spec_hash: Hash256,
    pub topology_policy: TopologyPolicy,
    pub collapse_policy: CollapsePolicy,
    pub operator_policy: OperatorBuildConfig,
    pub signature_policy: SignatureConfig,
    pub gate_policy: SignatureGateConfig,
    pub seed: u64,
    pub parent_blueprint_id: Option<Hash256>,
}
```

12.3 Search Policies

```
pub enum BlueprintSearchPolicy {
    TopK,
    Ucb,
    EpsilonGreedy,
    NoveltyWeighted,
    DeterministicGrid,
}
```

Alle Suchpolitiken **MUSS** deterministisch sein. Für zufallsähnliche Exploration **MUSS** ein Seed verwendet werden. Tie-Breaking **MUSS** über content-addressed IDs erfolgen.

12.4 Nichtdominierte Frontier

Die Suche **SOLLTE** eine nichtdominierte Frontier führen. Ein Blueprint a dominiert Blueprint b , wenn a in allen Zielmetriken mindestens gleich gut und in mindestens einer Zielmetrik besser ist.

Zulässige Zielmetriken:

- Gate-Pass-Rate,
- niedrige Fragmentierung,
- niedrige Degeneration,
- ausreichender Gap Score,
- niedrige Constraint-Verletzung,
- Replay-Identität,
- PSE-Commit-Outcome-Qualität,
- geringere Solver-Kosten.

13 Search Ledger

13.1 Funktion

Das Search Ledger ist ein experimentelles Audit-Log für Suchläufe. Es **MUSS** klar vom PSE-Crystal-Archiv getrennt bleiben. Es beweist Suchverlauf und Ranking-Entscheidungen, aber es beweist keine endgültige Wahrheit.

13.2 SearchLedgerEntry

Listing 13: SearchLedgerEntry

```
pub struct SearchLedgerEntry {
    pub entry_id: Hash256,
    pub parent_entry_id: Option<Hash256>,
    pub blueprint_id: Hash256,
    pub problem_spec_hash: Hash256,
    pub plan_id: Hash256,
    pub operator_id: Option<Hash256>,
    pub signature_id: Option<Hash256>,
    pub diagnostics_id: Option<Hash256>,
    pub gate_report_address: Hash256,
    pub commit_outcome_kind: CommitOutcomeKind,
    pub scores: BTreeMap<String, f64>,
    pub evidence_refs: Vec<EvidenceRef>,
    pub logical_index: u64,
}
```

13.3 Hash Chain

Jeder Eintrag **MUSS** den vorherigen Eintrag referenzieren, sofern er nicht der Genesis-Eintrag ist. Eine Ledger-Prüfung **MUSS** Parent-Ketten, Entry-IDs und kanonische Bytes prüfen.

14 Search Autopilot

14.1 Phasenmodell

Listing 14: AutopilotPhase

```
pub enum AutopilotPhase {
    Exploration,
    Exploitation,
    Refinement,
    Validation,
    Complete,
}
```

Phase	Aufgabe
Exploration	Breite Erzeugung unterschiedlicher Blueprint- und Operatorvarianten.
Exploitation	Fokus auf Varianten mit guten Gate-, Diagnose- und Replay-Scores.
Refinement	Feinanpassung von Schwellen, Gewichten, Collapse-Policies und Operatorparametern.
Validation	Replays, Cross-Seed-Checks, PSE-Bridge-Ausführung und Zertifikatsprüfung.
Complete	Erzeugung eines finalen SearchLedgerReports und optionaler CollapseCertificates.

14.2 AutopilotConfig

```
pub struct AutopilotConfig {
    pub max_cycles: u32,
```



```

pub max_evaluations: u32,
pub exploration_ratio: f64,
pub target_gate_pass_rate: f64,
pub require_replay_identity: bool,
pub require_signature_gate: bool,
pub seed: u64,
}

```

14.3 Stop-Regeln

Der Autopilot **MUSS** stoppen, wenn eine der folgenden Bedingungen erfüllt ist:

1. max_cycles erreicht.
2. max_evaluations erreicht.
3. Validierungsziel erreicht.
4. Kein gültiger Blueprint mehr erzeugbar.
5. Gate- oder Replay-Invarianten werden verletzt.

Wall-Clock-Runtime **DARF NICHT** Teil des deterministischen Stop-Kriteriums sein. Eine CLI **DARF** zusätzlich einen äußeren Laufzeitabbruch anbieten, dieser **MUSS** aber als externer Abort im Report markiert werden.

15 Integration in Reports und Zertifikate

15.1 TraversalRunReport-Erweiterung

TraversalRunReport **MUSS** optional folgende Felder aufnehmen:

```

pub struct TraversalRunReportSignatureExtension {
pub blueprint_id: Option<Hash256>,
pub operator_id: Option<Hash256>,
pub signature_id: Option<Hash256>,
pub diagnostics_id: Option<Hash256>,
pub signature_gate_outcome_id: Option<Hash256>,
pub search_ledger_entry_id: Option<Hash256>,
}

```

Wenn Signaturdiagnostik aktiviert ist, **MUSS** der Report die referenzierten Objekte entweder einbetten oder über content-addressed Artefaktpfade verfügbar machen.

15.2 CollapseCertificate-Erweiterung

CollapseCertificate **MUSS** Signaturbefunde als EvidenceRefs aufnehmen können:

```

pub struct CollapseCertificateSignatureEvidence {
pub operator_id: Hash256,
pub signature_id: Hash256,
pub diagnostics_id: Hash256,
pub gate_outcome_id: Option<Hash256>,
}

```

15.3 GateReport-Erweiterung

GateReport **SOLLTE** einen neutralen Kanal für Zusatzdiagnosen erhalten:

```
pub struct GateDiagnosticChannel {  
    pub channel_name: String,  
    pub passed: bool,  
    pub address: Hash256,  
    pub reasons: Vec<String>,  
}
```

16 CLI-Anforderungen

Die bestehende CLI **SOLLTE** erweitert werden, ohne existierende Subcommands zu brechen.

Listing 15: CLI-Zielprofil

```
pse-traverse-cli inspect <problem.json>  
pse-traverse-cli plan <problem.json> [--signature]  
pse-traverse-cli run <problem.json> [--signature] [--signature-gate]  
pse-traverse-cli replay <run_report.json>  
pse-traverse-cli search <problem.json> --max-evals N --signature
```

16.1 Subcommand: plan

Wenn `-signature` gesetzt ist, **MUSS** der Planreport zusätzlich Operator-, Signatur- und Diagnoseadressen ausgeben, sofern der Plan ohne Candidate-Ausführung profilierbar ist.

16.2 Subcommand: run

Wenn `-signature-gate` gesetzt ist, **MUSS** ein nicht bestandenenes Signature Gate bei `fail_closed=true` einen Commit verhindern.

16.3 Subcommand: search

`search` **MUSS** mehrere TraversalBlueprints erzeugen, evaluieren, ledgern und einen finalen SearchLedgerReport schreiben.

17 Determinismus und Canonicalization

17.1 Kanonische Serialisierung

Alle neuen Artefakte **MUSS** vor Hashing mit derselben kanonischen Strategie serialisiert werden, die im bestehenden System für Reports und Deskriptoren verwendet wird. Falls JCS bereits im Workspace verwendet wird, **MUSS** JCS beibehalten werden.

17.2 Sortierpflicht

Folgende Felder **MUSS** sortiert werden:

- Maps: BTreeMap statt HashMap.
- Mengen: vor Serialisierung als sortierte Vektoren.
- Kanten: kanonische Knotenordnung.
- Diagnosen: Nachrichten und Reasons nach stabiler Regel, sofern nicht Reihenfolge semantisch ist.
- Ledger-Einträge: nach `logical_index` und Entry-ID.

17.3 Seeds

Jeder Suchlauf **MUSS** einen Seed im `TraversalRunDescriptor` oder `TraversalBlueprint` speichern. Kein Modul **DARF** implizit Systemzufall verwenden.

18 Fehlersemantik

18.1 Fehlertypen

```
pub enum SignatureError {
    EmptyOperator,
    NonFiniteValue,
    QuantizationFailed,
    AlgorithmDidNotConverge,
    UnsupportedOperatorKind,
}

pub enum OperatorBuildError {
    EmptyGraph,
    InvalidNodeReference,
    InvalidWeight,
    MissingConstraintReference,
}
```

18.2 Fehlerauswirkung

Fehler	Wirkung
OperatorBuildError	Signatur-Schicht deaktiviert diesen Pfad; Gate entscheidet gemäß Config.
SignatureError	Bei fail-closed: Gate-Fail. Bei non-fail-closed: Diagnosekanal mit Fehler.
Nichtfinite Werte	Immer Fail oder Skip mit explizitem Grund; niemals Hash-Input.
Nichtkonvergenz	Diagnose-Fail mit Algorithmusname und Iterationslimit.
Ledger-Hashbruch	Replay-Fail.

19 Testanforderungen

19.1 Unit Tests

Die Implementierung **MUSS** mindestens folgende Unit Tests enthalten:

1. `operator_from_empty_graph_fails`
2. `operator_id_is_stable`
3. `operator_edges_are_sorted`
4. `signature_is_content_addressed`
5. `signature_quantization_is_stable`
6. `signature_rejects_nan_and_inf`
7. `diagnostic_scores_in_unit_interval`
8. `regime_uncoupled_for_fragmented_graph`

9. `regime_directed_for_asymmetric_graph`
10. `signature_gate_fail_closed_blocks_commit`
11. `signature_gate_non_fail_closed_does_not_block_commit`
12. `blueprint_id_is_stable`
13. `blueprint_search_is_deterministic_under_seed`
14. `search_ledger_hash_chain_verifies`
15. `search_ledger_detects_parent_mismatch`

19.2 Integration Tests

Die Implementierung **MUSS** mindestens folgende Integration Tests enthalten:

1. CLI `plan -signature` erzeugt byte-identische Reports bei zwei Läufen.
2. CLI `run -signature-gate` erzeugt bei Gate-Fail keinen Commit.
3. CLI `search -signature` erzeugt ein verifizierbares Ledger.
4. `replay` validiert Report, Signaturadressen und Ledgeradressen.
5. Bestehende `pse-traverse`-Tests bleiben unverändert grün.

20 Definition of Done

Ein Patch erfüllt PSE-TRAVERSE-SIGNATURE-01 erst, wenn alle folgenden Punkte erfüllt sind:

ID	Bedingung
D1	Alle neuen öffentlichen Typen sind dokumentiert.
D2	Der Kern enthält keine Begriffe <code>zeta</code> , <code>riemann</code> , <code>theta</code> , <code>RH</code> , <code>GUE</code> , <code>GOE</code> , <code>Poisson</code> in öffentlichen Typen, Configs, CLI-Flags oder Reportfeldern.
D3	Alle neuen Artefakte sind content-addressed.
D4	Zwei identische Runs erzeugen byte-identische Reports.
D5	Signaturdiagnostik kann deaktiviert werden, ohne bestehende Pipeline zu brechen.
D6	Fail-closed Signature Gate verhindert Commit.
D7	Non-fail-closed Signature Gate erscheint als Diagnose, ohne Commit automatisch zu verhindern.
D8	Search Ledger verifiziert seine Hash Chain.
D9	Search Autopilot ist deterministisch unter gleichem Seed.
D10	Kein Netzwerkzugriff im Kern.
D11	Keine Python-, NumPy- oder SciPy-Abhängigkeit im Rust-Kernel.
D12	<code>cargo test -workspace</code> läuft erfolgreich.
D13	Bestehende PSE-Commit-Brücke bleibt einzige Quelle echter <code>SemanticCrystals</code> .
D14	Replay prüft Signatur- und Diagnoseadressen.
D15	Mindestens ein Benchmark zeigt, dass Signaturdiagnostik Plan-Ranking oder GateReport messbar beeinflusst.

21 Empfohlene Implementierungssequenz

21.1 Phase S1 - Typen und Canonicalization

1. `operator.rs`, `signature.rs`, `signature_diag.rs` anlegen.

2. `SignatureValue` und Quantisierung implementieren.
3. Content Addressing für Operator, Signature und Diagnostics implementieren.
4. Unit Tests für Stabilität und Sortierung schreiben.

21.2 Phase S2 - Operatorbildung aus DoFGraph

1. Laplacian-Profil aus DoFGraph ableiten.
2. Directed Coupling Profile aus gerichteten Couplings ableiten.
3. Constraint-Potential aus Constraint-Gewichten ableiten.
4. Fehler für leere oder inkonsistente Graphen implementieren.

21.3 Phase S3 - Signaturdiagnostik

1. Minimal-Signaturprofil erzeugen.
2. Scores berechnen.
3. RegimeHint klassifizieren.
4. Diagnostik in ReportExtension einbetten.

21.4 Phase S4 - Signature Gate

1. `SignatureGateConfig` und `SignatureGateOutcome` implementieren.
2. GateReport-Kanal anbinden.
3. Fail-closed und non-fail-closed Tests schreiben.

21.5 Phase S5 - Blueprint Search und Ledger

1. `TraversalBlueprint` implementieren.
2. deterministische Suchpolitiken implementieren.
3. `SearchLedgerEntry` und Ledger-Verifikation implementieren.
4. CLI `search` ergänzen.

21.6 Phase S6 - Autopilot und Benchmark

1. Autopilot-Phasen implementieren.
2. Benchmark-Szenario ergänzen.
3. Nachweisen, dass Signaturdiagnostik Ranking oder Gate beeinflusst.
4. Replay-End-to-End absichern.

22 Beispiel: neutrale Problemverarbeitung

22.1 Input

Ein `ProblemSpec` beschreibt drei Dimensionen mit Couplings und Constraints. Die Signatur-Schicht liest keine Namen als Bedeutung. Sie benutzt nur Struktur:

```
Dimension A --hard--> Dimension B
Dimension B --soft--> Dimension C
Dimension A --directed--> Dimension C
```

22.2 Operator

Der Builder erzeugt ein Profil:

```
Laplacian:          symmetric dependency weights
DirectedCoupling:   A -> C asymmetry
ConstraintPotential: hard/soft weights per node
```

22.3 Diagnose

Mögliche Diagnose:

```
fragmentation_score = 0.0
degeneracy_score    = 0.33
asymmetry_score      = 0.42
gap_score            = 0.71
rigidity_score       = 0.62
regime_hint          = DirectedCoupled
```

22.4 Gate-Wirkung

Wenn der Plan einen kausal gerichteten Raum wie einen symmetrischen Raum behandeln würde, **SOLLTE** die Diagnose den Plan abstrafen oder eine kausal geordnete CollapsePolicy empfehlen.

23 Sicherheits- und Missbrauchsgrenzen

Die Signatur-Schicht ist ein Strukturdiagnosewerkzeug. Sie **DARF NICHT** sicherheitskritische Entscheidungen allein treffen. In sicherheitsrelevanten Domänen **MUSS** ein Domain-Verifier oder Human-Review-Gate vorgeschaltet oder nachgeschaltet werden.

PathExcision bleibt verbindlich: Wenn eine ProblemSpec operative, gefährliche oder unerwünschte Handlungsdetails enthält, **MUSS** die bestehende Excision-Logik greifen, bevor Solver- oder Oracle-Kontexte erzeugt werden.

24 Publikations- und Benennungsdisziplin

Das Modul **MUSS** neutral benannt werden. Zulässige Namen:

- pse-traverse-signature
- operator-signature traversal
- signature mining layer
- structural signature diagnostics

Unzulässige Kernnamen:

- zeta-traverse
- riemann-layer
- theta-miner
- gue-gate
- hilbert-polya-core

25 Zusammenfassung der Implementierungslogik

Die Spezifikation reduziert den aus ZetaLab extrahierten Mechanismus auf seine neutrale Maschine:

Nicht Zahlen oder Buchstaben lesen, sondern die Struktur vermessen, die sie tragen.

Für `pse-traverse` bedeutet das:

1. Ein Problemraum wird nicht nur zerlegt, sondern spektral profiliert.
2. Ein CollapsePlan wird nicht nur syntaktisch geprüft, sondern global strukturell bewertet.
3. GateReports erhalten zusätzliche, messbare Strukturdiagnosen.
4. Suchläufe werden nicht nur ausgeführt, sondern als Blueprint-Varianten exploriert, verglichen und geloggt.
5. Die PSE-Commit-Brücke bleibt epistemischer Endpunkt für echte Crystals.

Damit wird das System nicht domänenspezifischer, sondern allgemeiner: Jede spätere Spezialdomäne ist ein Adapter. Der Kern bleibt Struktur, Operator, Signatur, Diagnose, Gate und Evidenz.

A Agenten-Patch-Briefing

Der folgende Block kann einem Coding-Agenten als kompakte Anweisung dienen. Er ersetzt nicht die Spezifikation, verweist aber auf deren verbindliche Regeln.

PATCH-ID: PSE-TRAVERSE-SIGNATURE-01

Implement a domain-agnostic Operator-Signature Traversal Layer in `crates/pse-traverse`. Extract only the neutral machinery implied by the prior ZetaLab analysis: structural operators, signatures, diagnostics, signature gates, blueprint search, search ledger and deterministic search autopilot.

MUST:

- add `operator.rs`, `signature.rs`, `signature_diag.rs`, `signature_gate.rs`, `blueprint_search.rs`, `search_ledger.rs`, `search_autopilot.rs`
- content-address `StructuralOperator`, `Signature`, `SignatureDiagnostics`, `SignatureGateOutcome`, `TraversalBlueprint` and `SearchLedgerEntry`
- derive operators from `DoFGraph` without domain assumptions
- keep all maps sorted via `BTreeMap` and all vectors canonicalized
- integrate signature diagnostics into `GateReport`, `TraversalRunReport` and `CollapseCertificate` by address/reference
- provide fail-closed and non-fail-closed signature gate behavior
- add deterministic CLI support for `--signature`, `--signature-gate` and `search`
- preserve PSE bridge as the only `SemanticCrystal` creation path
- pass cargo test `--workspace`

MUST NOT:

- introduce `zeta`, `riemann`, `theta`, `RH`, `GUE`, `GOE`, `Poisson` as public core terms
- add Python, NumPy or SciPy dependencies to the Rust kernel
- use network access
- use wall-clock time in content-addressed IDs
- fabricate `SemanticCrystal`

Definition of Done is Section 18 of the LaTeX specification.